Project: Development of a Game-Based Environment for Reinforcement Learning.

1. Project Description

The goal of this project is to develop a video game environment and train an autonomous agent to play it using a Deep Reinforcement Learning (DRL) algorithm. The game is designed as an interactive environment in which the agent must observe the game state, select appropriate actions, and learn a near-optimal policy through trial and error.

From an RL perspective, the game is modeled as a sequential decision-making problem (Markov Decision Process). At each time step, the agent receives an observation of the current game state and selects an action from a predefined action space. The environment then updates according to the selected action and returns a reward reflecting the quality of the decision. The agent's objective is to learn a policy that maximizes the expected cumulative reward.

2. Instructions

1. Build your game using a suitable package or framework, such as Pygame or any other appropriate package
2. The game must support two different modes:
    - **Interactive mode**: a graphical interface allowing a human player to view and play the game.
    - **Headless mode**: no graphical interface, used for training and testing the RL agent or other models**.**
3. Train an RL agent using a Deep Reinforcement Learning (DRL) algorithm of your choice.
4. Analyze training convergence
    - Check convergence using the total reward per episode (not discounted reward).
    - Save the total reward per episode in a TensorBoard log.
    - Using Average episode reward for given sliding window may be nessessary when very fluctuate
5. Design and implement a rule-based agent to play the game
6. Compare the trained RL agent and the rule-based agent
    - Use multiple appropriate evaluation metrics of your choice and justify your selection.
    - Save the evaluation results in a CSV file.
    - Display the corresponding plots and tables in the Jupyter notebook.

3. Report content

The report must be a presentation (slides) in PDF format and include:

1. Brief description of the game
2. Detailed RL problem formulation (MDP): reward function, state definition, action space.
3. Clearly describe and justify the selected RL algorithm, network architecture, and main hyperparameters.

4. Clearly describe the rules and decision-making strategy used by the rule-based baseline agent.
5. Code description:
    1. Working environment: machine specifications and main Python packages (with versions)
    2. Project structure: content of each file, and locations of plots, saved models, and evaluation files (CSV and logs)
    3. How to run each file
6. Present and discuss the results using appropriate tables and/or plots
7. Resources and references

4. Source code format

1. The project must follow the **uv** project format (**pyproject.toml, uv.lock**), so that it can be reproduced with **uv sync**.
2. The project must contain the following files:
    1. **train.py**: trains the RL agent.
    2. **test.py**: evaluates the RL agent and the rule-based agent.
    3. **play.py**: lets a human play the game.
    4. **observe.py**: visualizes a single episode played by the trained agent (or the rule-based agent).
    5. **evaluate.ipynb**: displays the plots and tables.
    6. Include a **README.md** with setup and execution instructions.

5. Important Insights

- The submission deadline is 31/12/2026. After this deadline, points will be deducted.
- You will also be graded on the perfection of your work.
- Submit your work (code + report) on the eLearn platform; a dedicated space will be created for this purpose.
- You are allowed to use the Stable-Baselines3 framework or any other alternative framework that provides pre-implemented RL algorithms. However, you must use a customized neural network architecture rather than a predefined architecture.
- You are not required to design a completely new game. You may use an existing game or game concept; however, you must add your own original contributions or modifications. You must also clearly provide and cite the resources and references that inspired your game design.
- You are allowed to use AI tools; however, they should be used as assistive tools and should not be used to complete the project on your behalf.
- Each group must consist of 5 to 6 members.

6. Useful Resources and Notes

- You can follow this link to get an idea about the utility and how to use TensorBoard: https://www.youtube.com/watch?v=k7KfYXXrOj0"
- link to stable-baselines framework: https://github.com/DLR-RM/stable-baselines3